

INFORMATIONWEEK WEBINAR · JUN 2026

Governing AI where the work actually happens

A practical look at governing AI access, for the people and the agents now acting on their behalf.

Eytan Lerba · Product Lead, Governance & Infrastructure · Retool

Who I am, and what Retool does

Me

Eytan Lerba. I lead the Governance & Infrastructure product at Retool. 15+ years building, including Stripe, Google, YouTube, and a startup I co-founded. I spend my days on exactly this: access, permissions, and now agents.

Retool

Where teams build internal software, AI apps, and agents, on their own data, in their own environment. That vantage point, across a wide range of teams putting AI into production, is where today's view comes from.

I'll keep this a practitioner's view. The principles hold whatever you build on.

THE GAP

Everyone agrees governance should enable AI. The hard part is enforcing it where the work happens. At runtime.

The visibility gap **is** the governance gap

We asked 300+ CTOs, CIOs, and CISOs how governed their internal AI really is. The honest answers:

8%

call their internal governance “strong”

5%

are very confident they see what's running in production

55%

want controls in one platform only 7% want them per-app

Source: Retool, [State of AI Governance 2026](#)

Governing AI comes down to **three questions**

It always reduces to access and action: who can reach what, and what they can do. These three get you there. Permissions are the foundation under all of them.

1 · Who's acting?

A human, an agent using someone's identity, or an agent with its own. You govern each differently.

2 · What can they reach?

Which data and actions, and where that limit is actually enforced.

3 · Can you stay in control?

Can you see what happened, verify it before it acts, and pay for it without surprises?

QUESTION 1 OF 3

Who's acting?

A human, an agent using someone's identity, or an agent with its own, and why that's the first thing to pin down.

Three kinds of **identity** touch your data

And each is governed differently. Most platforms collapse all three into one over-privileged account.

1

Human

Acts as themselves. Scoped to the role you already gave them.

2

Agent · as the user

Uses the person's own OAuth and permissions. Acts “as you,” at machine speed.

3

Agent · its own service user

Its own identity & service account, never a human's. Powerful, so it must be explicitly scoped.

The highest-risk identity

You can't scope, audit, or trust an action until you know which of these took it.

QUESTION 2 OF 3

What can they reach?

Which data, which actions, and the part that decides everything: where that limit is actually enforced.

Three layers of **access control**

Define access once at the Platform layer; it's enforced at runtime inside every Application. Never per app.

1 **Environment**

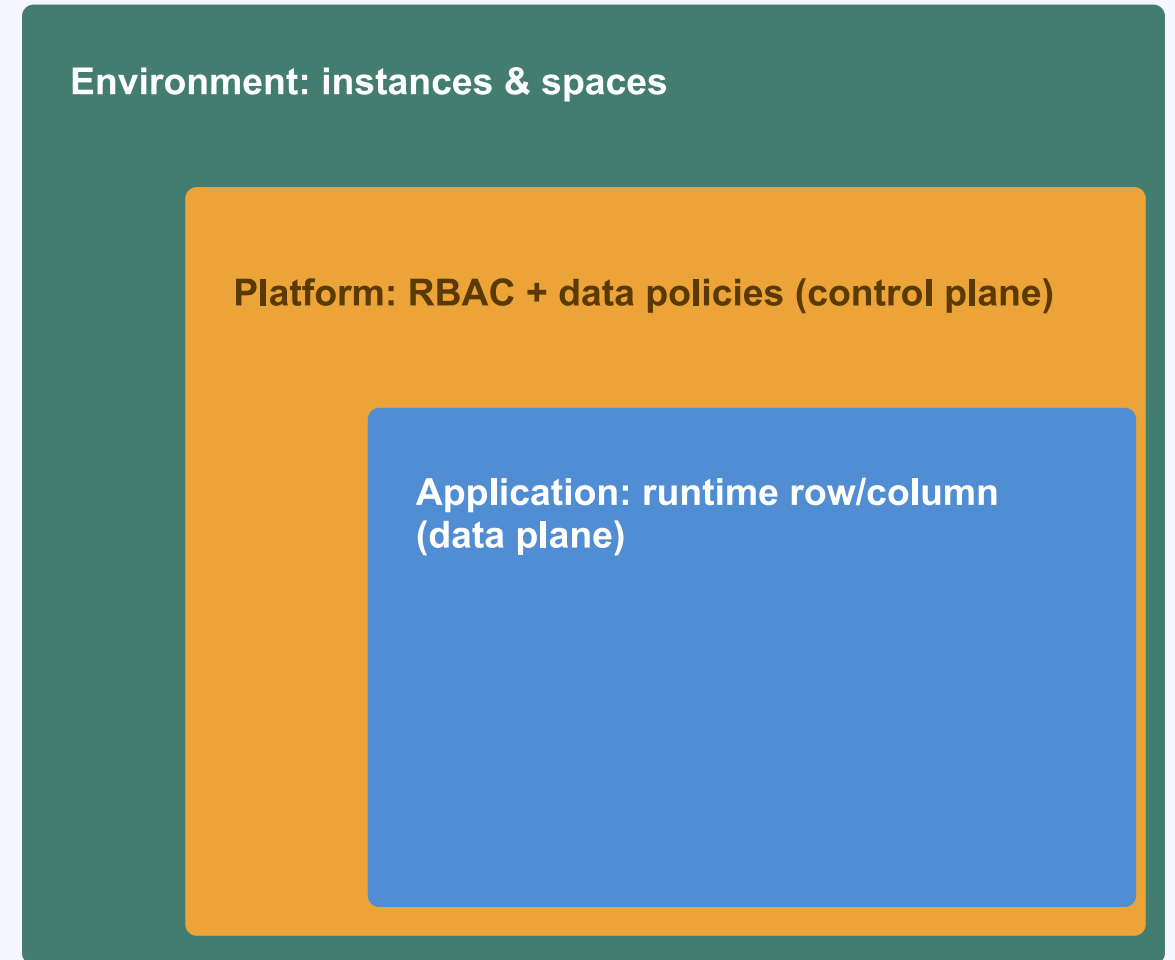
Where it runs; who's isolated. Instances & spaces.

2 **Platform (control plane)**

Define once: RBAC + data policies (rows, columns, masking).

3 **Application (data plane)**

Enforced at runtime, every request. Not authored here.



Scoped delegation: more secure AND more usable

Least privilege isn't just an admin setting. It's a dial you set at every level.

Admins

Split god-mode into scopes. Let the design team own branding without touching SSO or spaces.

Builders

Scope which resources and data each builder can build against, not the whole warehouse.

End users

Scope what they see and do inside an app, down to the row, column, and button.

We expected a tradeoff between control and speed. Scoping at every level gave us both.

More secure: least privilege by default, small blast radius. · More usable: every level gets exactly what it owns, no waiting on a central admin.

Write the rule once. It runs on every request.

Instead of burying rules in each app, set them once on the serving side, enforced for every app, automation, and agent. A few examples:

The screenshot shows the 'PostgreSQL / new policy' interface with three numbered callouts:

- 1** (Define access): A list of tables on the left. 'employee_details' is selected, showing '3 rules'. Below it, 'table_1', 'table_2', 'table_3', and 'table_4' are listed with a red 'Denied' status.
- 2** (Describe policy): The 'Columns denied' section. It shows a table with columns: ID, Email, Name, Role, Comp, Manager. The 'Comp' column is highlighted as denied.
- 3** (Assign groups & environments): The 'Row access' section. It shows a table with columns: ID, Email, Name, Role, Comp, Manager. The 'Role' column is set to 'Design' and the 'Manager' column is set to 'Susan Boyle'.

ID	Email	Name	Role	Comp	Manager
1					
2	alice@gmail.com	Alice Smith	Marketing		John Mayer
3	james@gmail.com	James Brown	Engineering		Emily Clark
4					
5	robert@gmail.com	Robert Wilson	Product		Janet Leigh
6	emma@example.com	Emma Taylor	Sales		David Bowie
7	michael@example.com	Michael Johnson	Marketing		Sarah Connor
8					
9					
10	olivia@example.com	Olivia Wright	Legal		Bruce Wayne
11	charles@example.com	Charles King	Design		Janet Leigh

1 Block the table

Whole tables denied unless you're HR or Finance

2 Mask columns

Comp and other sensitive fields hidden from reads

3 Row-level security

Rows filtered by role, so reps see only their own

Examples reflect Retool's direction for serving-side policy; availability varies by capability.

QUESTION 3 OF 3

Can you stay in control?

Seeing what happened, verifying it before it acts, and paying for it without surprises.

Governing the **agent** at runtime

What we keep hearing from enterprise governance teams, and how we're building for it.

Deterministic, not vibes

You don't trust an LLM to enforce permissions. It will say “whoops, I did it anyway.” Hard rules run on the serving side. AI helps you author policy, not enforce it.

A guardian agent checks intent in context

Where we're headed: before an agent acts, an independent compliance agent reviews its plan against your policies and logs an allow/deny with a reason.

Trust, but verify

Audit every action, export to your SIEM, run evals, and escalate to a human at consequential steps. Reactive becomes proactive.

Two more ways to **stay in control**

Beyond who-can-reach-what: keep sensitive data in place, and keep spend from running away.

Data stays put

Sensitive data never leaves your perimeter, even while agents act. Per-user OAuth (no shared accounts) and masking apply at request time. Self-hosting supports the data-residency and sovereignty posture that regulations like GDPR increasingly push toward.

Spend stays bounded

An agent that can act can also spend. Without limits, teams can blow through an AI budget in months. Set per-project budgets and usage observability on AI and compute. Govern the dollars the way you govern the data.

Three questions to **take back to your stack**

Ask these of whatever you build on. Clean answers mean you're in control. Gaps are your roadmap.

ASK YOUR OWN STACK

YOU'RE IN CONTROL IF...

MONDAY

1 · Who's acting?

you can name the identity behind every action, human or agent.

Inventory which identities your agents use.

2 · What can they reach?

access is layered and enforced on the serving side, not per app.

Move one sensitive resource's rules to the serving side.

3 · Can you stay in control?

it's deterministic, watched, and budgeted.

Put usage + cost on one dashboard; audit one workflow.

THE TAKEAWAY

Who's acting, what they can reach, and can you stay in control. Answer all three where the work happens.

Read the full data: Retool's State of AI Governance 2026 → retool.com/blog/ai-governance-report-2026

Eytan Lerba · Retool · [find me on LinkedIn](#)